

1. Introducción

El presente informe documenta la ejecución de pruebas unitarias realizadas sobre el sistema InvenTree, utilizado como proyecto base para el trabajo final del curso de Pruebas de Software. InvenTree es una aplicación web orientada a la gestión de inventario, partes, stock, órdenes y empresas, desarrollada principalmente con tecnologías de backend basadas en Python y Django.

Para el Hito 2, el equipo trabajó sobre el fork denominado InvenTree-VIUDANEGRA, tomando como base el repositorio original de InvenTree. La actividad principal de este informe corresponde a la ejecución de pruebas unitarias existentes en el proyecto, utilizando herramientas automatizadas como pytest y coverage.py.

A diferencia de las pruebas funcionales, que se ejecutan manualmente desde la interfaz de usuario, las pruebas unitarias se ejecutaron desde terminal sobre módulos específicos del backend. El propósito fue validar el comportamiento interno de componentes del sistema, registrar el estado de los casos ejecutados y medir la cobertura de código obtenida durante la ejecución.

1.1. Alcance

El alcance de este informe comprende la ejecución de pruebas unitarias automatizadas en un entorno local Linux, tomando como base módulos principales del backend de InvenTree.

Módulo	Descripción general
stock	Gestión de existencias, ubicaciones y movimientos de inventario
part	Gestión de partes, categorías y componentes
order	Gestión de órdenes relacionadas al flujo de inventario
company	Gestión de empresas, proveedores y fabricantes

La ejecución se realizó usando pytest y coverage.py, con una base de datos SQLite configurada para pruebas locales. Asimismo, se aplicaron migraciones antes de la ejecución para evitar errores de base de datos pendientes.

Quedan fuera del alcance de esta ejecución:

- pruebas funcionales manuales de caja negra;
- pruebas de interfaz de usuario;
- pruebas automatizadas de frontend;
- pruebas de rendimiento;
- pruebas de seguridad;
- ejecución completa de todos los módulos del backend;
- ejecución en ambiente CI/CD completo.

Por tanto, los resultados presentados corresponden a una ejecución parcial de pruebas unitarias del backend, no a una validación total del sistema.

1.2. Responsables de la actividad

Integrante	Rol en la actividad
Jose Jaita	Elaboración y estructuración del informe de pruebas unitarias
Christopher Del Castillo	Ejecución de pruebas unitarias, configuración del entorno, generación de evidencias y reporte de cobertura
Equipo Viuda Negra	Revisión, consolidación y validación del entregable

La ejecución técnica fue realizada en el entorno local configurado por el responsable de ejecución, generando archivos de evidencia para respaldar los resultados obtenidos.

1.3. Referencias

- Repositorio original de InvenTree: <https://github.com/inventree/InvenTree>
- Repositorio del equipo: <https://github.com/jrolando19/InvenTree-VIUDANEGRA>
- Documentación oficial de InvenTree: <https://docs.inventree.org/>
- Documentación de Django Testing.
- Documentación de pytest.
- Documentación de coverage.py.
- Plantilla de Plan de Pruebas proporcionada en el curso.
- Plan de Pruebas Unitarias elaborado por el equipo para el Hito 2.

2. Entorno de pruebas

2.1. Configuración del entorno

Las pruebas unitarias fueron ejecutadas en un entorno local Linux, utilizando un entorno virtual de Python y una base de datos SQLite configurada específicamente para pruebas.

Componente	Detalle
Sistema operativo	Pop! OS / Linux
Ruta local del proyecto	~/Documentos/InvenTree
Entorno virtual	.venv
Framework de pruebas	pytest
Herramienta de cobertura	coverage.py
Versión de pytest verificada	pytest 7.4.4
Versión de coverage verificada	Coverage.py 7.14.1
Versión de Django verificada	Django 6.0.6
Base de datos de prueba	SQLite3
Módulos ejecutados	stock, part, order, company
Evidencias	Carpeta evidencias ruparupa/issue 24/

La ejecución se realizó desde terminal, sin depender obligatoriamente del entorno Docker. Para las pruebas unitarias se empleó el entorno virtual local, mientras que Docker fue utilizado en otras actividades del Hito 2 para levantar el sistema y preparar datos funcionales.

2.2. Preparación del entorno

Antes de la ejecución final, fue necesario preparar el entorno local y corregir varios errores de configuración. Inicialmente se recreó el entorno virtual debido a problemas en una configuración anterior. Luego se instalaron las dependencias necesarias del proyecto.

```
cd ~/Documentos/InvenTree
rm -rf .venv
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip setuptools wheel
pip install -e "[dev]"
```

Posteriormente se verificaron las herramientas principales:

```
pytest --version
coverage --version
python -c "import django; print(django.get_version())"
```

Durante la preparación se identificaron errores de configuración que impidieron ejecutar las pruebas en un primer momento. Estos errores fueron corregidos antes de la ejecución final.

Entre los problemas solucionados estuvieron:

- falta de dependencia structlog;
- ausencia de configuración para INVENTREE_STATIC_ROOT;
- falta de configuración de base de datos mediante INVENTREE_DB_ENGINE;
- ausencia de INVENTREE_SITE_URL o INVENTREE_TRUSTED_ORIGINS;
- migraciones pendientes en la base de datos SQLite de prueba.

2.3. Limitaciones del entorno

La ejecución presenta las siguientes limitaciones:

- se ejecutó una selección parcial de módulos del backend;
- no se ejecutó la suite completa del proyecto original;
- la cobertura obtenida corresponde al entorno local, no al pipeline oficial del proyecto;
- algunos errores pueden estar asociados a configuración local, datos de prueba o dependencias del entorno;
- no se levantó un ambiente CI/CD completo para reproducir exactamente el comportamiento del repositorio original;
- los resultados no deben interpretarse como cobertura total de todo InvenTree.

Estas limitaciones son importantes porque explican por qué la cobertura obtenida no alcanza el porcentaje esperado de 85% y por qué se registraron fallos y errores durante la ejecución.

3. Ejecución de pruebas unitarias

3.1. Comandos utilizados

La ejecución final se realizó desde la ruta del backend del proyecto. Antes de ejecutar las pruebas, se crearon carpetas de soporte y se exportaron las variables necesarias para que InvenTree pudiera iniciar correctamente en modo local.

```
cd ~/Documentos/InvenTree
source .venv/bin/activate
cd src/backend

mkdir -p ../../test_static ../../test_media ../../test_backup ../../test_db

export INVENTREE_STATIC_ROOT="$HOME/Documentos/InvenTree/test_static"
export INVENTREE_MEDIA_ROOT="$HOME/Documentos/InvenTree/test_media"
export INVENTREE_BACKUP_DIR="$HOME/Documentos/InvenTree/test_backup"
export INVENTREE_SECRET_KEY="test-secret-key-ruparupa"
export INVENTREE_DB_ENGINE="sqlite3"
export
INVENTREE_DB_NAME="$HOME/Documentos/InvenTree/test_db/inventree_test.sqlite3"
export INVENTREE_SITE_URL="http://localhost"
export INVENTREE_TRUSTED_ORIGINS="http://localhost,http://127.0.0.1"
```

Luego se aplicaron las migraciones correspondientes a la base de datos SQLite de prueba:

```
python InvenTree/manage.py migrate | tee
../../evidencias_ruparupa/issue_24/15_migraciones_sqlite_test.txt
```

Después se ejecutaron las pruebas unitarias con coverage:

```
coverage erase
coverage run -m pytest InvenTree/stock InvenTree/part InvenTree/order InvenTree/company -v |
tee ../../evidencias_ruparupa/issue_24/16_resultado_pytest_coverage_post_migrate.txt
```

Finalmente, se generó el reporte de cobertura:

```
coverage report | tee ../../evidencias_ruparupa/issue_24/17_reporte_cobertura_post_migrate.txt
```

3.2. Evidencias generadas

Como parte de la trazabilidad técnica, se generaron archivos de evidencia mediante el uso de tee. Esto permitió registrar la salida de los comandos directamente en archivos de texto.

Evidencia	Archivo	Descripción
Búsqueda de pruebas unitarias	01_pruebas_unitarias_encontradas.txt	Registro de archivos de prueba encontrados en el proyecto
Migraciones SQLite	15_migraciones_sqlite_test.txt	Salida de la ejecución de migraciones en la base de datos de prueba
Resultado de ejecución	16_resultado_pytest_coverage_post_migrate.txt	Resultado completo de pytest con pruebas aprobadas, fallidas, errores y advertencias
Reporte de cobertura	17_reporte_cobertura_post_migrate.txt	Reporte generado por coverage report con cobertura global de 74%

```

FAILED InvenTree/part/test_part.py::PartNotificationTest::test_low_stock_notification
FAILED InvenTree/part/test_pricing.py::PartPricingTests::test_bom_pricing - A...
FAILED InvenTree/part/test_pricing.py::PartPricingTests::test_internal_pricing
FAILED InvenTree/part/test_pricing.py::PartPricingTests::test_multi_level_bom
FAILED InvenTree/part/test_pricing.py::PartPricingTests::test_supplier_part_pricing
FAILED InvenTree/order/test_api.py::PurchaseOrderDownloadTest::test_download_csv
FAILED InvenTree/order/test_api.py::PurchaseOrderDownloadTest::test_download_line_items
FAILED InvenTree/order/test_api.py::SalesOrderTest::test_export - ValueError:...
FAILED InvenTree/order/test_api.py::SalesOrderDownloadTest::test_download_csv
FAILED InvenTree/order/test_api.py::SalesOrderDownloadTest::test_download_xlsx
FAILED InvenTree/order/test_api.py::ReturnOrderTests::test_export - ValueErro...
FAILED InvenTree/company/test_migrations.py::TestParameterMigrations::test_data_migration
FAILED InvenTree/company/test_migrations.py::TestManufacturerField::test_company_objects
FAILED InvenTree/company/test_migrations.py::TestManufacturerPart::test_manufacturer_part_objects
FAILED InvenTree/company/test_migrations.py::TestCurrencyMigration::test_currency_migration
FAILED InvenTree/company/test_migrations.py::TestAddressMigration::test_address_migration
FAILED InvenTree/company/test_migrations.py::TestSupplierPartQuantity::test_supplier_part_quantity
FAILED InvenTree/company/test_migrations.py::TestManufacturerPartParameterMigration::test_manufacturer_part_parameter_migration
ERROR InvenTree/order/tests.py::OrderTest::test_add_items - part.models.Part...
ERROR InvenTree/order/tests.py::OrderTest::test_basics - part.models.Part.Doe...
ERROR InvenTree/order/tests.py::OrderTest::test_locking - part.models.Part.Do...
ERROR InvenTree/order/tests.py::OrderTest::test_metadata - part.models.Part.D...
ERROR InvenTree/order/tests.py::OrderTest::test_new_po_notification - part.mo...
ERROR InvenTree/order/tests.py::OrderTest::test_on_order - part.models.Part.D...
ERROR InvenTree/order/tests.py::OrderTest::test_overdue - part.models.Part.Do...
ERROR InvenTree/order/tests.py::OrderTest::test_overdue_notification - part.m...
ERROR InvenTree/order/tests.py::OrderTest::test_pricing - part.models.Part.Do...
ERROR InvenTree/order/tests.py::OrderTest::test_rebuild_reference - part.mode...
ERROR InvenTree/order/tests.py::OrderTest::test_receive - part.models.Part.Do...
ERROR InvenTree/order/tests.py::OrderTest::test_receive_convert_currency - pa...
ERROR InvenTree/order/tests.py::OrderTest::test_receive_pack_size - part.mode...
ERROR InvenTree/order/tests.py::OrderTest::test_validate_contact - part.model...
ERROR InvenTree/order/tests.py::OrderTest::test_validate_dates - part.models....
===== 24 failed, 487 passed, 143 warnings, 15 errors in 4559.47s (1:15:59) =====
(.venv) ftgy@pop-os:~/Documentos/InvenTree/src/backends$ coverage report | tee ../evidencias_ruparupa/issue_24/17_reporte_cobertura_post_migrate.txt
Name                                                                    Stmts   Miss    Cover
-----
InvenTree/InvenTree/__init__.py                                           0         0     100%
InvenTree/InvenTree/admin.py                                              9         0     100%
InvenTree/InvenTree/api.py       404      114      72%
InvenTree/InvenTree/api_version.py                                       3         0     100%
InvenTree/InvenTree/apps.py       174        56      68%
InvenTree/InvenTree/auth_overrides.py                                   116        76      34%
InvenTree/InvenTree/backends.py     83        41      51%
InvenTree/InvenTree/backup.py      89        49      45%
InvenTree/InvenTree/cache.py       73        21      71%

```

InvenTree/stock/migrations/0117_stockitemtracking_part_alter_stockitemtracking_item.py	5	0	100%
InvenTree/stock/migrations/0118_auto_20260205_1218.py	34	12	65%
InvenTree/stock/migrations/0119_alter_stockitemtestresult_date.py	5	0	100%
InvenTree/stock/migrations/__init__.py	0	0	100%
InvenTree/stock/models.py	1183	247	79%
InvenTree/stock/serializers.py	586	58	90%
InvenTree/stock/status_codes.py	42	0	100%
InvenTree/stock/tasks.py	49	28	43%
InvenTree/stock/test_api.py	1084	20	98%
InvenTree/stock/test_migrations.py	161	0	100%
InvenTree/stock/test_views.py	59	0	100%
InvenTree/stock/tests.py	809	23	97%
InvenTree/users/__init__.py	0	0	100%
InvenTree/users/admin.py	49	7	86%
InvenTree/users/api.py	203	80	61%
InvenTree/users/apps.py	25	2	92%
InvenTree/users/authentication.py	19	9	53%
InvenTree/users/migrations/0001_initial.py	6	0	100%
InvenTree/users/migrations/0002_auto_20201004_0158.py	4	0	100%
InvenTree/users/migrations/0003_auto_20201005_2227.py	4	0	100%
InvenTree/users/migrations/0004_auto_20210113_1909.py	4	0	100%
InvenTree/users/migrations/0005_owner_model.py	5	0	100%
InvenTree/users/migrations/0006_alter_ruleset_name.py	4	0	100%
InvenTree/users/migrations/0007_alter_ruleset_name.py	5	0	100%
InvenTree/users/migrations/0008_apitoken.py	7	0	100%
InvenTree/users/migrations/0009_auto_20231020_2356.py	4	0	100%
InvenTree/users/migrations/0010_alter_apitoken_key.py	6	0	100%
InvenTree/users/migrations/0011_auto_20240523_1640.py	7	0	100%
InvenTree/users/migrations/0012_alter_ruleset_can_view.py	4	0	100%
InvenTree/users/migrations/0013_migrate_mfa_20240408_1659.py	21	7	67%
InvenTree/users/migrations/0014_userprofile.py	11	1	91%
InvenTree/users/migrations/0015_alter_userprofile_type.py	4	0	100%
InvenTree/users/migrations/__init__.py	0	0	100%
InvenTree/users/models.py	241	49	80%
InvenTree/users/oauth2_scopes.py	11	0	100%
InvenTree/users/permissions.py	68	5	93%
InvenTree/users/ruleset.py	24	1	96%
InvenTree/users/serializers.py	173	66	62%
InvenTree/users/tasks.py	85	16	81%
InvenTree/web/__init__.py	0	0	100%
InvenTree/web/templatetags/__init__.py	0	0	100%
InvenTree/web/templatetags/spa_helper.py	36	22	39%
InvenTree/web/urls.py	8	0	100%

TOTAL	42077	11059	74%

4. Resultados de ejecución

4.1. Resumen general

La ejecución de pruebas unitarias finalizó a nivel de proceso, aunque no todos los casos fueron exitosos. El resultado obtenido fue el siguiente:

Métrica	Resultado
Pruebas aprobadas	487
Pruebas fallidas	24
Errores	15
Advertencias	143
Total aproximado de pruebas ejecutadas	526
Tiempo total de ejecución	4559.47 segundos
Tiempo aproximado	1 h 15 min 59 s
Cobertura global obtenida	74%

El total aproximado de pruebas ejecutadas se obtiene considerando las pruebas aprobadas, fallidas y con error:

$$487 + 24 + 15 = 526$$

4.2. Interpretación de resultados

Los resultados muestran que la mayoría de pruebas ejecutadas fueron aprobadas. De un total aproximado de 526 pruebas, 487 pasaron correctamente. Sin embargo, también se registraron 24 pruebas fallidas y 15 errores, por lo que la ejecución local no puede considerarse completamente satisfactoria.

La tasa aproximada de pruebas aprobadas fue:

$$487 / 526 \approx 92.6\%$$

Este porcentaje indica que una parte importante de la suite ejecutada tuvo comportamiento correcto. Sin embargo, los fallos y errores deben analizarse antes de afirmar que los módulos evaluados se encuentran completamente validados.

La presencia de errores no implica necesariamente defectos reales en el código de producción. En proyectos grandes como InvenTree, algunos errores pueden deberse a configuración incompleta del entorno local, datos de prueba no cargados, diferencias entre el entorno local y el entorno CI/CD, dependencias o servicios secundarios no disponibles, o configuraciones específicas requeridas por algunos tests.

4.3. Cobertura obtenida

La cobertura global obtenida fue de 74%, según el reporte generado mediante coverage report.

Stmts	Miss	Cover
42077	11059	74%

El reporte de cobertura registró un total de 42077 sentencias evaluadas, de las cuales 11059 no fueron cubiertas por la ejecución. Como resultado, la cobertura global obtenida fue de 74%.

Indicador	Resultado
Cobertura esperada	85%
Cobertura obtenida	74%
Diferencia respecto a la meta	-11%

Aunque la cobertura obtenida no alcanza el 85% esperado, representa una medición real de la ejecución local realizada sobre los módulos seleccionados del backend. Para mejorar este valor se recomienda analizar los fallos y errores detectados, corregir incidencias de configuración y ampliar la ejecución hacia más módulos del proyecto.

5. Incidencias detectadas durante la ejecución

5.1. Catálogo de incidencias

ID	Incidencia	Descripción	Estado
I-01	Dependencia faltante	Se presentó error por ausencia de structlog	Corregido
I-02	Falta de ruta para estáticos	El sistema requería definir INVENTREE_STATIC_ROOT	Corregido
I-03	Configuración de base de datos	El sistema requería INVENTREE_DB_ENGINE e INVENTREE_DB_NAME	Corregido
I-04	Falta de URL del sitio	Se requirió configurar INVENTREE_SITE_URL y INVENTREE_TRUSTED_ORIGINS	Corregido
I-05	Migraciones pendientes	La base SQLite de prueba requería migraciones	Corregido
I-06	Pruebas fallidas	Se obtuvieron 24 pruebas fallidas	Pendiente de análisis
I-07	Errores de ejecución	Se obtuvieron 15 errores durante la ejecución	Pendiente de análisis
I-08	Cobertura menor a la esperada	Se obtuvo 74%, por debajo del 85% esperado	Pendiente de mejora

Los fallos y errores se reportan como incidencias de ejecución. No todos deben interpretarse inmediatamente como defectos reales del sistema, ya que pueden estar relacionados con configuración local, datos de prueba o diferencias con el entorno de integración continua.

5.2. Detalle de pruebas fallidas

La siguiente tabla resume las 24 pruebas que finalizaron con estado FAILED en la ejecución local.

ID	Módulo	Prueba fallida	Observación
F-01	stock	InvenTree/stock/test_api.py::StockItemListTest::test export	Exportación de stock
F-02	stock	InvenTree/stock/test_api.py::StockItemListTest::test large export	Exportación masiva de stock
F-03	stock	InvenTree/stock/tests.py::TestResultTest::test duplicate item tests	Validación de resultados duplicados
F-04	part	InvenTree/part/test_api.py::PartOptionsAPITest::test part label translation	Traducción de etiquetas de partes
F-05	part	InvenTree/part/test_api.py::PartAPITest::test part download	Descarga de datos de partes
F-06	part	InvenTree/part/test_api.py::BomItemTest::test bom export	Exportación de lista de materiales
F-07	part	InvenTree/part/test_part.py::PartNotificationTest::test low stock notification	Notificación de bajo stock
F-08	part	InvenTree/part/test_pricing.py::PartPricingTests::test bom pricing	Cálculo de precios de BOM
F-09	part	InvenTree/part/test_pricing.py::PartPricingTests::test internal pricing	Cálculo de precios internos
F-10	part	InvenTree/part/test_pricing.py::PartPricingTests::test multi level bom	Cálculo en BOM multinivel
F-11	part	InvenTree/part/test_pricing.py::PartPricingTests::test supplier part pricing	Precio de parte de proveedor
F-12	order	InvenTree/order/test_api.py::PurchaseOrderDownloadTest::test download csv	Descarga CSV de orden de compra
F-13	order	InvenTree/order/test_api.py::PurchaseOrderDownloadTest::test download line items	Descarga de ítems de orden
F-14	order	InvenTree/order/test_api.py::SalesOrderTest::test export	Exportación de orden de venta
F-15	order	InvenTree/order/test_api.py::SalesOrderDownloadTest::test download csv	Descarga CSV de orden de venta
F-16	order	InvenTree/order/test_api.py::SalesOrderDownloadTest::test download xlsx	Descarga XLSX de orden de venta
F-17	order	InvenTree/order/test_api.py::ReturnOrderTests::test export	Exportación de orden de retorno
F-18	part	InvenTree/part/test_migrations.py::TestParameterMigrations::test data migration	Migración de parámetros
F-19	company	InvenTree/company/test_migrations.py::TestManufacturerField::test company objects	Migración de campo fabricante
F-20	company	InvenTree/company/test_migrations.py::TestManufacturerPart::test_manufacturer_part_objects	Migración de partes de fabricante
F-21	company	InvenTree/company/test_migrations.py::TestCurrencyMigration::test_currency_migration	Migración de moneda

F-22	company	InvenTree/company/test_migrations.py::TestAddressMigration::test_address_migration	Migración de direcciones
F-23	company	InvenTree/company/test_migrations.py::TestSupplierPartQuantity::test_supplier_part_quantity	Cantidad de parte de proveedor
F-24	company	InvenTree/company/test_migrations.py::TestManufacturerPartParameterMigration::test_manufacturer_part_parameter_migration	Migración de parámetros de parte de fabricante

5.3. Detalle de errores

Los 15 errores se concentran en pruebas del módulo order, específicamente en la clase OrderTest. Deben revisarse con la salida completa de pytest para determinar la causa exacta.

ID	Módulo	Prueba con error
E-01	order	InvenTree/order/tests.py::OrderTest::test_add_items
E-02	order	InvenTree/order/tests.py::OrderTest::test_basics
E-03	order	InvenTree/order/tests.py::OrderTest::test_locking
E-04	order	InvenTree/order/tests.py::OrderTest::test_metadata
E-05	order	InvenTree/order/tests.py::OrderTest::test_new_notification
E-06	order	InvenTree/order/tests.py::OrderTest::test_on_order
E-07	order	InvenTree/order/tests.py::OrderTest::test_overdue
E-08	order	InvenTree/order/tests.py::OrderTest::test_overdue_notification
E-09	order	InvenTree/order/tests.py::OrderTest::test_pricing
E-10	order	InvenTree/order/tests.py::OrderTest::test_rebuild_reference
E-11	order	InvenTree/order/tests.py::OrderTest::test_receive
E-12	order	InvenTree/order/tests.py::OrderTest::test_receive_convert_currency
E-13	order	InvenTree/order/tests.py::OrderTest::test_receive_pack_size
E-14	order	InvenTree/order/tests.py::OrderTest::test_validate_contact
E-15	order	InvenTree/order/tests.py::OrderTest::test_validate_dates

En la salida de ejecución se observa que estos errores están relacionados con referencias a modelos del módulo part, por lo que requieren revisión adicional para determinar si la causa corresponde a configuración

del entorno local, datos de prueba faltantes, dependencias entre módulos o comportamiento interno del sistema.

6. Técnicas y mecanismos técnicos identificados en la suite de pruebas

En esta versión se reemplaza la descripción convencional de técnicas de caja negra por un análisis más técnico de la suite realmente ejecutada. El objetivo no es afirmar que el equipo diseñó todos los casos desde cero, sino identificar cómo están construidas las pruebas automatizadas existentes de InvenTree y qué mecanismos verifican en los módulos stock, part, order y company.

6.1. Descubrimiento y selección de pruebas con pytest

La ejecución se realizó mediante pytest, invocando rutas concretas del backend: InvenTree/stock, InvenTree/part, InvenTree/order e InvenTree/company. Esta forma de ejecución permite que pytest descubra clases, funciones y archivos de prueba dentro de los módulos indicados, sin ejecutar necesariamente toda la suite del repositorio.

Técnicamente, esta selección permite delimitar el alcance de la medición de cobertura y concentrar el análisis en módulos directamente relacionados con inventario, partes, órdenes y empresas. Por ello, el 74% de cobertura debe interpretarse como resultado de una ejecución dirigida, no como cobertura total del proyecto completo.

6.2. Pruebas sobre modelos Django y reglas de dominio

Una parte importante de la suite valida objetos del ORM de Django. Estas pruebas crean, consultan, modifican y verifican instancias de modelos como partes, ítems de stock, empresas, proveedores y órdenes. El objetivo técnico es comprobar que las reglas del dominio se mantengan en la capa de modelos, antes de llegar a la interfaz de usuario.

En este nivel se verifican restricciones como existencia de relaciones entre entidades, consistencia de cantidades, integridad de campos obligatorios, actualización de atributos calculados y persistencia correcta de los cambios en la base de datos de prueba. Este tipo de prueba es relevante porque detecta errores en la lógica interna aunque no se interactúe con la interfaz gráfica.

6.3. Pruebas de API, serialización y exportación de datos

También se identifican pruebas orientadas a endpoints y operaciones de API, especialmente en archivos como test_api.py. Estas pruebas no solo verifican funciones aisladas, sino también respuestas HTTP, serialización de datos, descarga de archivos y exportación de información en formatos como CSV o XLSX.

Los fallos encontrados en casos como StockItemListTest::test_export, StockItemListTest::test_large_export, PurchaseOrderDownloadTest::test_download_csv, SalesOrderDownloadTest::test_download_xlsx y ReturnOrderTests::test_export muestran que una parte de la suite evalúa la generación de reportes y descargas desde el backend. Por tanto, estos casos son técnicamente más cercanos a pruebas de API/backend que a pruebas unitarias puras.

6.4. Pruebas de migraciones y compatibilidad de datos históricos

La suite incluye pruebas de migraciones, visibles en archivos como test_migrations.py. Estas pruebas verifican que los cambios en el esquema de base de datos y las transformaciones de datos históricos mantengan compatibilidad entre versiones.

En la ejecución se observaron fallos asociados a migraciones de company y part, por ejemplo TestManufacturerField, TestManufacturerPart, TestCurrencyMigration, TestAddressMigration, TestSupplierPartQuantity y TestManufacturerPartParameterMigration. Estos casos no validan una pantalla del sistema, sino la consistencia técnica de datos después de aplicar migraciones.

6.5. Pruebas de reglas de negocio complejas: BOM, pricing, stock y órdenes

Algunos tests cubren reglas de negocio que dependen de varias entidades relacionadas. Por ejemplo, los casos de pricing y BOM validan cálculos internos de costos, estructuras multinivel de lista de materiales, precios de proveedor y efectos sobre partes ensambladas.

En la ejecución se detectaron fallos en PartPricingTests::test_bom_pricing, PartPricingTests::test_internal_pricing, PartPricingTests::test_multi_level_bom y PartPricingTests::test_supplier_part_pricing. Estos casos son técnicamente importantes porque comprueban cálculos derivados, dependencia entre tablas y coherencia de precios a partir de datos relacionados.

También se ejecutaron pruebas relacionadas con stock y notificaciones, como PartNotificationTest::test_low_stock_notification. Este tipo de caso valida eventos internos o disparadores del sistema ante cambios de inventario, por lo que puede depender de configuración adicional del entorno.

6.6. Pruebas sensibles al entorno local

No todos los errores detectados deben interpretarse inmediatamente como defectos del producto. Parte de la suite depende de condiciones del entorno, como base de datos preparada, migraciones aplicadas, fixtures correctos, configuración de archivos, permisos, generación de documentos, servicios auxiliares o tareas asíncronas.

Los 15 errores concentrados en InvenTree/order/tests.py::OrderTest sugieren que el módulo order tiene dependencias internas con modelos del módulo part y con datos relacionados de compra, recepción, precios, contactos y fechas. Por ello, estos errores deben analizarse revisando la salida completa de pytest, antes de clasificarlos como defectos reales del sistema.

6.7. Matriz técnica de mecanismos probados

La siguiente matriz resume los mecanismos técnicos identificados en la ejecución de pruebas unitarias y de backend.

Mecanismo técnico	Archivos/casos representativos	Qué valida
ORM y modelos Django	stock/tests.py, part/test_part.py, order/tests.py	Persistencia, relaciones, reglas de dominio y consistencia de entidades
API y serialización	stock/test_api.py, part/test_api.py, order/test_api.py, company/test_api.py	Endpoints, respuestas HTTP, serialización y permisos de operaciones

Exportación/descarga	test_export, test_download_csv, test_download_xlsx	Generación de reportes, archivos CSV/XLSX y descargas desde backend
Migraciones	part/test_migrations.py, company/test_migrations.py	Compatibilidad de esquema y transformación de datos históricos
BOM y pricing	PartPricingTests, BomItemTest	Cálculo de costos, precios de proveedor, BOM multinivel y dependencias entre partes
Órdenes	OrderTest, PurchaseOrderDownloadTest, SalesOrderDownloadTest	Recepción, validación de fechas/contactos, estados y datos de órdenes
Notificaciones/eventos	PartNotificationTest::test_low_s tock_notification	Disparadores internos ante bajo stock u otros eventos del sistema

6.8. Síntesis técnica

La suite ejecutada combina pruebas unitarias, pruebas de componentes del backend y pruebas cercanas a integración ligera. Aunque se ejecutan desde pytest como parte del bloque de pruebas unitarias del proyecto, varias pruebas interactúan con la base de datos, modelos relacionados, endpoints internos y generación de archivos.

Por esta razón, los resultados obtenidos deben presentarse como ejecución de pruebas unitarias automatizadas del backend, con presencia de pruebas de componente e integración local. Esta precisión evita afirmar que todas las pruebas son unitarias puras y permite explicar por qué algunos fallos dependen del entorno o de datos relacionados.

7. Trazabilidad con GitHub Issues

La ejecución de pruebas unitarias se relaciona directamente con las tareas gestionadas en GitHub Issues. La trazabilidad permite demostrar que las actividades técnicas fueron registradas y vinculadas al avance del Hito 2.

Issue	Actividad relacionada	Estado
#18	Verificación del repositorio, versión y estructura	Completado
#19	Ejecución del proyecto mediante Docker Compose	Casi completado
#20	Preparación de datos de prueba y fixtures	Completado
#24	Re-ejecución de pruebas unitarias con cobertura	Ejecutado, pendiente de análisis detallado de fallos

El Issue #24 se vincula directamente con este informe, ya que corresponde a la ejecución de pruebas unitarias con pytest y coverage, así como a la generación del reporte de cobertura.

La evidencia técnica asociada a este issue se encuentra en la carpeta:

evidencias_ruparupa/issue_24/

8. Conclusiones y recomendaciones

8.1. Conclusiones

- La ejecución de pruebas unitarias permitió validar una parte importante del backend de InvenTree en un entorno local Linux.
- Se ejecutaron pruebas sobre los módulos stock, part, order y company, obteniendo 487 pruebas aprobadas, 24 fallidas, 15 errores y 143 advertencias.
- La cobertura global obtenida fue de 74%. Aunque este resultado no alcanza el 85% esperado, sí representa una evidencia técnica válida del estado de la suite de pruebas en el entorno local configurado.
- La ejecución no puede considerarse completamente satisfactoria debido a la presencia de fallos y errores. Sin embargo, estos resultados son útiles porque permiten identificar incidencias concretas que deben ser revisadas posteriormente.
- Durante la preparación del entorno se resolvieron varios problemas de configuración, como dependencias faltantes, rutas de archivos estáticos, configuración de base de datos, URL del sitio y migraciones pendientes.
- Los resultados obtenidos demuestran que la ejecución de pruebas unitarias fue realizada de forma automatizada y documentada mediante archivos de evidencia, cumpliendo con la trazabilidad solicitada para el Hito 2.

8.2. Recomendaciones

- Analizar individualmente las 24 pruebas fallidas y los 15 errores registrados, clasificándolos según su causa probable: configuración del entorno, datos de prueba faltantes, dependencias, diferencias con el entorno CI/CD o posibles defectos del sistema.
- Ejecutar nuevamente la suite luego de corregir las incidencias detectadas, con la finalidad de reducir la cantidad de fallos y acercarse a la cobertura mínima esperada del 85%.
- Ampliar la ejecución a más módulos del backend y validar si existen configuraciones adicionales necesarias para reproducir el entorno oficial del proyecto.
- Mantener los archivos de evidencia generados y enlazarlos desde la Wiki o desde los GitHub Issues correspondientes, especialmente el Issue #24.
- Conservar una sola versión consolidada de resultados en el informe oficial del equipo, evitando contradicciones entre documentos de Drive, Wiki y evidencias técnicas.